

Computer Vision

Homework-1

Bilal Ahmed
ahmedb@purdue.edu

Question 1

What are all the points in the representational space $\mathbb{R}^3$ that are the homogeneous coordinates of the origin in the physical space $\mathbb{R}^2$?

Answer: A point $(x, y) \in \mathbb{R}^2$ is represented in homogeneous coordinates as $\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$ or in general by the equivalence class $k \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$ for $k \in \mathbb{R}, k \neq 0$. Hence all the points specified by $k \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$ for $k \in \mathbb{R}, k \neq 0$ are valid representations for the origin in the physical space $\mathbb{R}^2$.

Question 2

Are all points at infinity in the physical plane $\mathbb{R}^2$ the same? Justify your answer.

Answer: Homogeneous coordinates (HC) provides an elegant way to represent points at infinity. As we know HC representation $\begin{bmatrix} u \\ v \\ w \end{bmatrix}$ can be converted back to points in $\mathbb{R}^2$ as $\begin{bmatrix} u/w \\ v/w \end{bmatrix}$. So as $w \rightarrow 0$ the corresponding point in physical space approaches infinity along the direction specified by u and v . So consider two points at infinity represented in HC as $\begin{bmatrix} u1 \\ v1 \\ 0 \end{bmatrix}$ and $\begin{bmatrix} u2 \\ v2 \\ 0 \end{bmatrix}$ and the line connecting these two points given by $\begin{bmatrix} u1 \\ v1 \\ 0 \end{bmatrix} \times \begin{bmatrix} u2 \\ v2 \\ 0 \end{bmatrix}$ represents a line at infinity and is parameterized as $k \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$ for $k \in \mathbb{R}, k \neq 0$. As this line at infinity ℓ_∞ passes through both the points $\begin{bmatrix} u1 \\ v1 \\ 0 \end{bmatrix}$ and $\begin{bmatrix} u2 \\ v2 \\ 0 \end{bmatrix}$, we can say that all such points at infinity in physical plane $\mathbb{R}^2$ would lie on ℓ_∞ . But it is important to note that different points in $\mathbb{R}^2$ approach infinity along different directions but end up on the same line ℓ_∞ (which seems difficult to comprehend). We call these points as ideal points.

Question 3

Prove that the matrix rank of a degenerate conic can never exceed 2.

Answer: A degenerate conic can be thought of as a curve formed when a plane intersects a double cone along the axis of the double cone. We can see that as just two intersecting straight

lines, say $\mathbf{l}$ and $\mathbf{m}$ and we can represent such a conic $\mathbf{C}$ in terms of lines $\mathbf{l}$ and $\mathbf{m}$ as follows:

$$\mathbf{C} = \mathbf{l}\mathbf{m}^T + \mathbf{m}\mathbf{l}^T$$

Each term is an outer product of two vectors, and has a rank of 1. Since the columns of the outer product would be scalar multiples of each other. Adding two matrices having rank 1 each, would give us a matrix with rank 2 or less. Rank would be 2 when column spaces of individual matrices are non-overlapping, otherwise 1. So matrix rank of degenerate matrix $\mathbf{C}$ can never exceed 1.

Question 4

A line in $\mathbb{R}^2$ is defined by two points. That raises the question how many points define a conic and a degenerate conic in $\mathbb{R}^2$? Justify your answer.

Answer: A line in $\mathbb{R}^2$ is written in implicit algebraic form as;

$$ax + by + c = 0$$

or equivalently using HC representation, as;

$$\mathbf{l}^T \mathbf{x} = 0$$

where $\mathbf{l} = \begin{bmatrix} a \\ b \\ c \end{bmatrix}$ parametrizes the line. We also know that any $k\mathbf{l}$, $k \in \mathbb{R}$ where $k \neq 0$ would represent the same line. So in order to completely characterize a line, we only need two parameters (e.g. two ratios a/c and b/c). We can solve for these two unknowns using only 2 points. Similarly, looking at the expression for the conic, we have;

$$ax^2 + bxy + cy^2 + dx + ey + f = 0$$

or equivalently,

$$\frac{a}{f}x^2 + \frac{b}{f}xy + \frac{c}{f}y^2 + \frac{d}{f}x + \frac{e}{f}y + 1 = 0$$

We only of 5 parameters (or 5 ratios) to find out. So we can completely define a conic using 5 points. We can also apply the thinking of representing conic by equivalence class $k\mathbf{C}$, $k \in \mathbb{R}$ where $k \neq 0$. That also brings us to the same conclusion of needing 5 ratios. For the case of degenerate conics, coefficients a and c would be 0, because we only have straight lines, so quadratic terms have to be zero. So equation reduces to;

$$bxy + dx + ey + f = 0$$

So we have 3 ratios to determine, meaning we can completely define a degenerate conic by 3 points.

Question 5

Derive in just 3 steps the intersection of two lines l_1 and l_2 with l_1 passing through the points (0,0) and (3,4), and with l_2 passing through the points (-1, 4) and (3/2, -1). How many steps would take you if the first line passed through (-1, 2) and (1, -2)?

Answer:

$$l_1 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \times \begin{bmatrix} 3 \\ 4 \\ 1 \end{bmatrix} = \begin{bmatrix} -4 \\ 3 \\ 0 \end{bmatrix}$$

$$l_2 = \begin{bmatrix} -1 \\ 4 \\ 1 \end{bmatrix} \times \begin{bmatrix} \frac{3}{2} \\ -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 5 \\ \frac{5}{2} \\ -5 \end{bmatrix} = \begin{bmatrix} 1 \\ \frac{1}{2} \\ -1 \end{bmatrix}$$

So the intersection point of these lines would be,

$$\mathbf{x} = \begin{bmatrix} -4 \\ 3 \\ 0 \end{bmatrix} \times \begin{bmatrix} 1 \\ \frac{1}{2} \\ -1 \end{bmatrix} = \begin{bmatrix} -3 \\ -4 \\ -5 \end{bmatrix}$$

In case when the first line l_1 passes through $(-1, 2)$ and $(1, -2)$;

$$l_1 = \begin{bmatrix} -1 \\ 2 \\ 1 \end{bmatrix} \times \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 2 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix}$$

we can see the first two values in both the lines have the same ratio, meaning that these are parallel lines. So we say right away that the lines would intersect at infinity i.e. ℓ_∞

Question 6

As you know, when a point p is on a conic C , the tangent to the conic at that point is given by $l = Cp$. That raises the question as to what Cp would correspond to when p was outside the conic. As you'll see later in class, when p is outside the conic, Cp is the line that joins the two points of contact if you draw tangents to C from the point p . This line is referred to as the polar line. Now let our conic C be an ellipse that is centered at the coordinates $(1, 4)$, with $a = 1$ and $b = 2$, where a and b , respectively, are the lengths of semi-minor and semi-major axes. For simplicity, assume that the minor axis is parallel to x-axis and the major axis is parallel to y-axis. Let p be the origin of the $\mathbb{R}^2$ physical plane. Find the intersections points of the polar line with x- and y-axes.

Answer: The equation of ellipse would be;

$$\frac{(x-1)^2}{1^2} + \frac{(y-4)^2}{2^2} = 1$$

$$4(x^2 + 1 - 2x) + y^2 + 16 - 8y = 4$$

$$4x^2 + y^2 - 8x - 8y + 16 = 0$$

We can represent this as;

$$C = \begin{bmatrix} 4 & 0 & -4 \\ 0 & 1 & -4 \\ -4 & -4 & 16 \end{bmatrix}$$

The polar line $l = Cp$ when p is origin;

$$l = \begin{bmatrix} 4 & 0 & -4 \\ 0 & 1 & -4 \\ -4 & -4 & 16 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -4 \\ -4 \\ 16 \end{bmatrix}$$

We can represent x-axis in HC as $\begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$,

$$\begin{bmatrix} -4 \\ -4 \\ 16 \end{bmatrix} \times \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} -16 \\ 0 \\ -4 \end{bmatrix} = \begin{bmatrix} 4 \\ 0 \\ 1 \end{bmatrix}$$

i.e. intersection of polar line with x-axis would be at (4,0). Similarly, y-axis in HC would be $\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$,

$$\begin{bmatrix} -4 \\ -4 \\ 16 \end{bmatrix} \times \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 16 \\ 4 \end{bmatrix} = \begin{bmatrix} 0 \\ 4 \\ 1 \end{bmatrix}$$

i.e. intersection of polar line with y-axis would be at (0,4).

Question 7

Lets say you are designing an arcade game where the user has to aim at random triangles displayed on the screen as shown in Fig. 1 . The user is sitting at origin and has a laser gun to aim at the incoming triangles. Given the aiming angle chosen by the user α along with the coordinates of the vertices of the triangle v_1, v_2, v_3 , how would you quickly compute if the user's aim is correct? Solve for the instance shown in the figure.

Answer: Here is the procedure to find out if the aiming laser would hit the target triangle.

1. convert aiming angle to line representation l_a
2. get representation of line l_{12} intersecting v_1 & v_2
3. get representation of line l_{23} intersecting v_2 & v_3
4. get representation of line l_{13} intersecting v_1 & v_3
5. Compute intersection point of l_a & l_{12} and get the physical coordinates, its x coordinate must lie within x coordinates of v_1 and v_2 .
6. Compute intersection point of l_a & l_{23} and get the physical coordinates, its x coordinate must lie within x coordinates of v_2 and v_3 .
7. Compute intersection point of l_a & l_{13} and get the physical coordinates, its x coordinate must lie within x coordinates of v_1 and v_3 .

If any of the 3 conditions in the last steps 5, 6, 7 is satisfied, the laser will intersect the triangle. Solving for the example shown,

1. Since the line is pointing at an angle α , there would lie a point $(1, \tan(\alpha))$ on the aiming line, intersecting this point with origin $(0, 0)$ would give us the expression of l_a . In the problem shown $\alpha = 45^\circ$, Using HC,

$$l_a = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \times \begin{bmatrix} 1 \\ \tan(\alpha) \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \times \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \\ 0 \end{bmatrix}$$

2.

$$l_{12} = \begin{bmatrix} 5 \\ 3 \\ 1 \end{bmatrix} \times \begin{bmatrix} 7 \\ 5 \\ 1 \end{bmatrix} = \begin{bmatrix} -2 \\ 2 \\ 4 \end{bmatrix}$$

3.

$$l_{23} = \begin{bmatrix} 3 \\ 5 \\ 1 \end{bmatrix} \times \begin{bmatrix} 7 \\ 5 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 4 \\ -20 \end{bmatrix}$$

4.

$$l_{13} = \begin{bmatrix} 3 \\ 5 \\ 1 \end{bmatrix} \times \begin{bmatrix} 5 \\ 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \\ -16 \end{bmatrix}$$

5.

$$x_1 = l_a \times l_{12} = \begin{bmatrix} -1 \\ 1 \\ 0 \end{bmatrix} \times \begin{bmatrix} -2 \\ 2 \\ 4 \end{bmatrix} = \begin{bmatrix} 4 \\ 4 \\ 0 \end{bmatrix}$$

intersects at infinity.

6.

$$x_1 = l_a \times l_{23} = \begin{bmatrix} -1 \\ 1 \\ 0 \end{bmatrix} \times \begin{bmatrix} 0 \\ 4 \\ -20 \end{bmatrix} = \begin{bmatrix} -20 \\ -20 \\ -4 \end{bmatrix} = \begin{bmatrix} 5 \\ 5 \\ 1 \end{bmatrix}$$

x coordinate = 5 that lies between 3 and 7.

7.

$$x_1 = l_a \times l_{13} = \begin{bmatrix} -1 \\ 1 \\ 0 \end{bmatrix} \times \begin{bmatrix} 2 \\ 2 \\ -16 \end{bmatrix} = \begin{bmatrix} -16 \\ -16 \\ -4 \end{bmatrix} = \begin{bmatrix} 4 \\ 4 \\ 1 \end{bmatrix}$$

x coordinate = 4 that lies between 3 and 5.

Since the conditions on step 6 and 7 are satisfied, the line will intersect the triangle.

Python simulations

I implemented this algorithm in Python and simulated for randomly generated triangles and aiming angles, Few cases are shown in the Fig. where 'green' color of triangle means laser would hit the target and 'red' means it would miss the target.

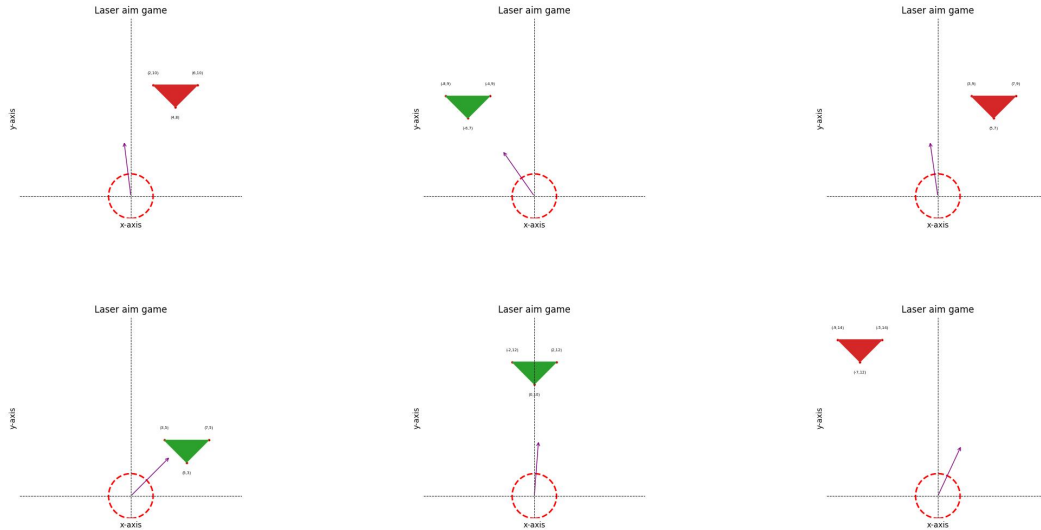


Figure 1: Simulations of laser aim game.

Python Code

```

1  import os
2  import math
3  import numpy as np
4  import computer_vision
5  from pathlib import Path
6  import matplotlib.pyplot as plt
7  import computer_vision.utils as utils
8  from computer_vision.utils import Plotter
9
10
11 def get_physical_coordinates(x_hc):
12     """Returns physical coordinates from homogeneous coordinates."""
13     x = x_hc[0]/x_hc[2]
14     y = x_hc[1]/x_hc[2]
15     return (x,y)
16
17 def laser_aim_check(vertices, alpha):
18     """Checks if the laser is going to intersect the triangle.
19
20     Args:
21         alpha: float = laser aiming angle in degrees.
22         vertices: (v1, v2, v3): where v1, v2, v3 are tuples themselves,
23             specifying physical (x,y) coordinates vertices of triangle.
24
25     Return:
26         bool: True when laser will intersect triangle, false otherwise
27     """
28     alpha_rad = math.radians(alpha)
29
30     origin = [0, 0, 1]
31     point_on_aiming_line = [1, math.tan(alpha_rad), 1]
32     aiming_line = np.cross(origin, point_on_aiming_line)
33     print(f"aiming_line: {aiming_line}")
34     v1, v2, v3 = vertices
35
36     # lines intersecting vertices of triangle.
37     l12 = np.cross([*v1, 1], [*v2, 1])
38     print(f"l12: {l12}")
39     l23 = np.cross([*v2, 1], [*v3, 1])
40     print(f"l23: {l23}")
41     l13 = np.cross([*v1, 1], [*v3, 1])
42     print(f"l13: {l13}")
43     # points of intersection of aiming line with lines connecting vertices.
44     x12 = np.cross(l12, aiming_line)
45     x23 = np.cross(l23, aiming_line)
46     x13 = np.cross(l13, aiming_line)
47     print(f"x12: {x12}")
48     print(f"x23: {x23}")
49     print(f"x13: {x13}")
50
51     x12_physical_x, *_ = get_physical_coordinates(x12)
52     x23_physical_x, *_ = get_physical_coordinates(x23)
53     x13_physical_x, *_ = get_physical_coordinates(x13)
54
55     # test if aiming line intersects the triangle or not...
56     if (x12_physical_x > v1[0] and x12_physical_x < v2[0]) or \

```

```
57         (x23_physical_x > v3[0] and x23_physical_x < v2[0]) or \
58         (x13_physical_x > v3[0] and x13_physical_x < v1[0]):
59     return True
60 else:
61     return False
62
63
64
65 def laser_aim_game():
66     """For randomly generated triangles and aiming angles, the function
67     checks if the laser is going to intersect the triangle and sets its
68     color to green
69     otherwise red.
70     """
71     path = Path(computer_vision.__file__)
72     images_dir = path.parents[1] / 'images' / 'hw1'
73
74     triangle_config = {
75         'area': 4,
76         'x_min': -8,
77         'x_max': 8,
78         'y_min': 7,
79         'y_max': 15,
80     }
81
82     arrow_config = {
83         'start_point': (0, 0), # Starting point of the arrow
84         'length': 5,
85         'angle': 45,
86     }
87
88     arena_config = {
89         'circle_radius': 2,
90         'x_lim': (-10, 10),
91         'y_lim': (-2, 16),
92     }
93
94     # randomly places triangle
95     vertices = utils.create_random_triangle_2d(**triangle_config)
96     # setting the laser aim randomly
97     alpha = 20 * np.random.randn() + 90
98
99     ax = Plotter.plot_arena(**arena_config)
100     # check if laser aim is good enough to hit the enemy triangle
101     hit = laser_aim_check(vertices=vertices, alpha=alpha)
102     if hit:
103         tri_color = 'tab:green'
104     else:
105         tri_color = 'tab:red'
106     arrow_config['angle'] = alpha
107     ax = Plotter.plot_arrow(**arrow_config, ax=ax)
108     ax = Plotter.plot_triangle(vertices=vertices, ax=ax, color=tri_color)
109
110     plt.savefig(os.path.join(images_dir, 'image6.jpg'))
111
112 if __name__ == '__main__':
113     # laser_aim_game()
114     vertices = (
```

```
115         (5,3),
116         (7,5),
117         (3,5)
118     )
119     alpha = 45
120     hit = laser_aim_check(vertices=vertices, alpha=alpha)
121     if hit:
122         print(f"It's is hit")
123     else:
124         print(f"It's a miss.")
```

Listing 1: Example Python code